Task 1: (Student Academic Data Cleaning)

Prompt:

Generate a Python script to clean and preprocess a student academic dataset by handling missing values, removing duplicates, standardizing text, and encoding categorical variables. Ensure the dataset is transformed into a structured format suitable for academic analysis.

Code:

```python
import pandas as pd
from sklearn.preprocessing import LabelEncoder
from IPython.display import display  # for table format

# Step 1: Create dataset and save as CSV
data = pd.DataFrame({
    "student_id": [1, 2, 2, 3, 4],
    "name": ["Rahul", "Sneha", "Sneha", "Arjun", "Kiran"],
    "marks": [85, None, None, 78, 90],
    "attendance": [90, 85, 85, None, 95],
    "department": ["CSE", "cse", "csv", None, "ECE"],
    "gender": ["Male", "Female", "Female", "Male", "Male"]
})

data.to_csv("students.csv", index=False)

# Step 2: Load dataset
data = pd.read_csv("students.csv")

print("Before Cleaning:")
display(data)  # TABLE FORMAT

# Step 3: Handle missing values
data["marks"] = data["marks"].fillna(data["marks"].mean())
data["attendance"] = data["attendance"].fillna(data["attendance"].mean())
data["department"] = data["department"].fillna("Unknown")

# Step 4: Remove duplicates
data = data.drop_duplicates(subset="student_id").copy()

# Step 5: Standardize text
data["department"] = data["department"].str.upper()

# Step 6: Encode categorical variables
le = LabelEncoder()
data["gender"] = le.fit_transform(data["gender"])
data["department"] = le.fit_transform(data["department"])

print("\nAfter Cleaning:")
display(data)  # TABLE FORMAT
```

Sample Input/Output:

Before Cleaning:

	student_id	name	marks	attendance	department	gender
0	1	Rahul	85.0	90.0	CSE	Male
1	2	Sneha	NaN	85.0	cse	Female
2	2	Sneha	NaN	85.0	csv	Female
3	3	Arjun	78.0	NaN	NaN	Male
4	4	Kiran	90.0	95.0	ECE	Male

After Cleaning:

	student_id	name	marks	attendance	department	gender
0	1	Rahul	85.000000	90.00	0	1
1	2	Sneha	84.333333	85.00	0	0
3	3	Arjun	78.000000	88.75	2	1
4	4	Kiran	90.000000	95.00	1	1

Code Explanation:

1. This task demonstrates preprocessing of a student academic dataset using Python.
2. A sample dataset is first created and saved as a CSV file to simulate real-world data.
3. Missing values in marks and attendance are replaced using mean values.
4. Duplicate records are removed based on student ID to ensure data consistency.
5. Department names are standardized to maintain uniform formatting.
6. Categorical variables like gender and department are encoded numerically.
7. The cleaned dataset becomes structured and ready for analysis or modeling.
8. Proper preprocessing improves data quality and ensures reliable results.

Task 2: (Financial Transaction Data Preprocessing)

Prompt:

Generate a Python script to preprocess a financial transaction dataset by handling date formats, creating new features, and normalizing numerical values. Ensure the processed dataset is suitable for further financial analysis and machine learning applications.

Code:

```python
import pandas as pd
from sklearn.preprocessing import MinMaxScaler
from IPython.display import display

# Step 1: Create dataset and save as CSV
data = pd.DataFrame({
    "transaction_id": [101, 102, 103, 104],
    "user_id": [1, 2, 1, 3],
    "amount": [500, 1500, 700, 2000],
    "transaction_date": ["2023-01-15", "2023-02-10", "2023-03-05", "2023-04-20"]
})

data.to_csv("transactions.csv", index=False)

# Step 2: Load dataset
data = pd.read_csv("transactions.csv")

print("Before Processing:")
display(data)

# Step 3: Convert to datetime
data["transaction_date"] = pd.to_datetime(data["transaction_date"])

# Step 4: Feature engineering (extract month & year)
data["year"] = data["transaction_date"].dt.year
data["month"] = data["transaction_date"].dt.month

# Step 5: Normalize amount column
scaler = MinMaxScaler()
data["amount"] = scaler.fit_transform(data[["amount"]])

print("\nAfter Processing:")
display(data)
```

Sample Input/Output:

Before Processing:

	transaction_id	user_id	amount	transaction_date
0	101	1	500	2023-01-15
1	102	2	1500	2023-02-10
2	103	1	700	2023-03-05
3	104	3	2000	2023-04-20

After Processing:

	transaction_id	user_id	amount	transaction_date	year	month
0	101	1	0.000000	2023-01-15	2023	1
1	102	2	0.666667	2023-02-10	2023	2
2	103	1	0.133333	2023-03-05	2023	3
3	104	3	1.000000	2023-04-20	2023	4

Code Explanation:

1. This task focuses on preprocessing financial transaction data for analysis.
2. The dataset is created and stored as a CSV file to simulate real-world input.
3. The transaction_date column is converted into datetime format for time-based operations.
4. New features such as year and month are extracted from the date column.
5. Feature engineering helps in analyzing transaction trends over time.
6. MinMaxScaler is used to normalize the transaction amount values between 0 and 1.
7. The processed dataset becomes suitable for machine learning models.
8. This improves data consistency and enables efficient financial analysis.

Task 3: (Environmental Sensor Data Preparation)

Prompt:

Generate a Python script to preprocess environmental sensor data by handling missing values, removing outliers, and normalizing numerical features. Ensure the dataset is cleaned and structured for environmental monitoring and analysis.

Code:

```
import pandas as pd
import numpy as np
from sklearn.preprocessing import MinMaxScaler
from IPython.display import display

# Step 1: Create dataset and save as CSV
data = pd.DataFrame({
    "sensor_id": [1, 2, 3, 4, 5],
    "temperature": [30, 35, None, 120, 28],   # 120 is outlier
    "humidity": [60, None, 55, 65, 70],
    "air_quality": [100, 110, None, 500, 90]  # 500 is outlier
})

data.to_csv("sensor_data.csv", index=False)

# Step 2: Load dataset
data = pd.read_csv("sensor_data.csv")

print("Before Processing:")
display(data)

# Step 3: Handle missing values
data["temperature"] = data["temperature"].fillna(data["temperature"].mean())
data["humidity"] = data["humidity"].fillna(data["humidity"].mean())
data["air_quality"] = data["air_quality"].fillna(data["air_quality"].mean())

# Step 4: Remove outliers using IQR method
for col in ["temperature", "humidity", "air_quality"]:
    Q1 = data[col].quantile(0.25)
    Q3 = data[col].quantile(0.75)
    IQR = Q3 - Q1
    data = data[(data[col] >= Q1 - 1.5 * IQR) & (data[col] <= Q3 + 1.5 * IQR)]

# Step 5: Normalize values
scaler = MinMaxScaler()
data[["temperature", "humidity", "air_quality"]] = scaler.fit_transform(
    data[["temperature", "humidity", "air_quality"]]
)

print("\nAfter Processing:")
display(data)
```

Sample Input/Output:

Before Processing:

	sensor_id	temperature	humidity	air_quality
0	1	30.0	60.0	100.0
1	2	35.0	NaN	110.0
2	3	NaN	55.0	NaN
3	4	120.0	65.0	500.0
4	5	28.0	70.0	90.0

After Processing:

	sensor_id	temperature	humidity	air_quality
0	1	0.285714	0.00	0.5
1	2	1.000000	0.25	1.0
4	5	0.000000	1.00	0.0

Code Explanation:

1. This task focuses on preprocessing environmental sensor data for analysis.
2. Missing values in temperature, humidity, and air quality are replaced using mean values.
3. Outliers are detected and removed using the Interquartile Range (IQR) method.
4. Removing outliers improves data quality and prevents incorrect analysis.
5. MinMaxScaler is used to normalize sensor readings between 0 and 1.
6. Normalization ensures uniform scale across all features.
7. The cleaned dataset becomes suitable for environmental monitoring systems.
8. This process enhances accuracy and reliability of sensor-based predictions.

Task 4: (Real-Time Application: Smart City Traffic Data Cleaning)

Prompt:

Generate a Python script to clean and preprocess smart city traffic data by standardizing location names, handling missing values, removing duplicates, and normalizing traffic metrics. Also generate a summary comparing dataset quality before and after cleaning.

Code:

```
import pandas as pd
from sklearn.preprocessing import MinMaxScaler
from IPython.display import display

# Step 1: Create dataset (simulate real traffic data)
data = pd.DataFrame({
    "sensor_id": [1, 2, 2, 3, 4],
    "road_name": ["MG Road", "my road", "my road", "Highway", "HIGHWAY"],
    "location": ["City Center", "city center", "city center", "Suburb", "suburb"],
    "traffic_density": [50, None, None, 90, 98],
    "speed": [40, 35, 35, 100, 45],
    "vehicle_count": [200, 100, 100, 300, 220]
})

data.to_csv("traffic_data.csv", index=False)

# Step 2: Load dataset
traffic = pd.read_csv("traffic_data.csv")

print("Before Cleaning:")
display(traffic)

print("\nSummary Before Cleaning:")
print(traffic.describe())

# Step 3: Standardize text (road & location)
traffic["road_name"] = traffic["road_name"].str.upper()
traffic["location"]  traffic["location"].str.upper()

# Step 4: Handle missing values
traffic["traffic_density"]  traffic["traffic_density"].fillna(traffic["traffic_density"].mean())

# Step 5: Remove duplicates
traffic = traffic.drop_duplicates().copy()

# Step 6: Normalize numerical columns
scaler = MinMaxScaler()
traffic[["speed", "vehicle_count"]]  scaler.fit_transform(
    traffic[["speed", "vehicle_count"]]
)

print("\nAfter Cleaning:")
display(traffic)

print("\nSummary After Cleaning:")
print(traffic.describe())
```

Sample Input/Output:

Before Cleaning:

	sensor_id	road_name	location	traffic_density	speed	vehicle_count
0	1	MG Road	City Center	NaN	40	200
1	2	mg road	city center	NaN	35	130
2	2	mg road	city center	NaN	35	130
3	3	Highway	Suburb	80.0	100	500
4	4	HIGHWAY	suburb	90.0	45	220

Summary Before Cleaning:

	sensor_id	traffic_density	speed	vehicle_count
count	5.000000	3.000000	5.00000	5.000000
mean	2.000000	71.666667	57.00000	256.000000
std	1.140175	28.816660	27.70379	137.404512
min	1.000000	50.000000	35.00000	130.000000
25%	1.000000	65.000000	35.00000	130.000000
50%	2.000000	80.000000	40.00000	200.000000
75%	3.000000	85.000000	45.00000	220.000000
max	4.000000	90.000000	100.00000	500.000000

After Cleaning:

	sensor_id	road_name	location	traffic_density	speed	vehicle_count
0	1	MGROAD	CITYCENTER	50.000000	HIGHWAY	0.1905
1	2	MGROAD	CITY CENTER	75.333333	0.000000	0.0000
3	3	HIGHWAY	SUBURB	80.000000	1.000000	1.0000
4	4	HIGHWAY	SUBURB	90.000000	0.153846	0.1250

Summary After Cleaning:

	sensor_id	traffic_density	speed	vehicle_count
count	4.000000	4.000000	4.000000	4.000000
mean	2.500000	73.333333	0.303692	0.290875
std	1.290994	16.096712	0.461792	0.473520
min	1.000000	50.000000	0.000000	0.000000
25%	1.750000	67.500000	0.053692	0.046875
50%	2.500000	76.666667	0.115385	0.093750
75%	3.250000	82.500000	0.365385	0.343750
max	4.000000	90.000000	1.000000	1.000000

Code Explanation:

1. This task demonstrates real-time traffic data preprocessing in a smart city system.
2. Text standardization ensures consistency in road and location names.
3. Missing traffic density values are replaced using mean values.
4. Duplicate sensor records are removed to avoid redundant data.
5. Numerical features like speed and vehicle count are normalized using MinMaxScaler.
6. Dataset summaries before and after cleaning highlight improvements in data quality.

7. Cleaned data becomes suitable for traffic analysis and prediction systems.
8. This process enhances accuracy and reliability of smart city applications.

Task 5: (Social Media Text Data Cleaning and Feature Extraction)

Prompt:

Generate a Python script to clean and preprocess social media text data by removing noise, handling duplicates, and performing NLP techniques. Extract useful features such as word count and sentiment score for analysis.

Code:

```python
import pandas as pd
import re
from IPyth[illegible]display import display

# NLP libraries
import nltk
from nltk.corpus import stopwords
from nltk.stem.[illegible] import PorterStemmer
from textblob import TextBlob

# Download required resources
nltk.download('stopwords')

# Step 1: Create larger dataset
data = pd.DataFrame({
    "post_id": [1,2,3,4,5,6,7,8,9,10],
    "text": [
        "I love this product! http://example.com",
        "Worst service ever!!!",
        "Worst service ever!!!",   # duplicate
        None,                      # null
        "Amazing experience :) #happy",
        "Not satisfied with the quality...",
        "Super fast delivery!!!",
        "Terrible support, very disappointed",
        "Check this out https://link.com great stuff",
        "Okay product, not bad",
```

Sample Input/Output:

```
Before Cleaning:
[nltk_data] Downloading package stopwords to /root/nltk_data...
[nltk_data]   Package stopwords is already up-to-date!
```

	post_id	text
0	1	I love this product! http://example.com
1	2	Worst service ever!!!
2	3	Worst service ever!!!
3	4	NaN
4	5	Amazing experience :) #happy
5	6	Not satisfied with the quality...
6	7	Super fast delivery!!!
7	8	Terrible support, very disappointed
8	9	Check this out https://link.com great stuff
9	10	Okay product, not bad

After Cleaning & Feature Extraction:

	post_id	text	cleaned_text	word_count	sentiment
0	1	I love this product! http://example.com	love product	2	0.600000
1	2	Worst service ever!!!	worst service ever	3	-1.000000
2	3	Worst service ever!!!	worst service ever	3	-1.000000
4	5	Amazing experience :) #happy	amazing experience happy	3	0.000000
5	6	Not satisfied with the quality...	satisfied quality	2	0.000000
6	7	Super fast delivery!!!	super fast delivery	3	0.269667
7	8	Terrible support, very disappointed	terrible support disappointed	3	0.000000
8	9	Check this out https://link.com great stuff	check great stuff	3	0.800000
9	10	Okay product, not bad	okay product bad	3	-0.100000

Code Explanation:

1. This task focuses on preprocessing social media text data for sentiment analysis.
2. Null and duplicate posts are removed to ensure data quality.
3. Regular expressions are used to remove URLs and unwanted characters.
4. Text is converted to lowercase and tokenized into words.
5. Stopwords are removed and stemming is applied to reduce words to root form.

6. Word count feature is extracted to measure text length.
7. Sentiment polarity is calculated using TextBlob for analysis.
8. The processed dataset becomes suitable for NLP and machine learning models.